

Tập 0: Setup Home Lab Chuẩn Chuyên Gia

Lý thuyết: Chọn đúng vũ khí trước khi xuất trận

Network Thực Chiến · Series: Kubernetes Networking · Tập 00

Câu hỏi đặt ra

"Mình là Network Engineer, mình chỉ cần cài Minikube hoặc kind lên máy là học K8s Networking được rồi, đúng không?"

Câu trả lời: KHÔNG.

Và trong tập này, chúng ta sẽ tìm hiểu tại sao.



Mục tiêu của một Network Engineer khi học K8s

Kỹ sư lập trình chỉ cần Kubernetes **chạy** ứng dụng.

Network Engineer cần Kubernetes để **hiểu sâu về mạng**:

- Quan sát interface ảo `veth` được tạo ra như thế nào.
- Bắt gói tin thực tế bằng `tcpdump` trên card mạng vật lý của Node.
- Xem bảng `iptables` / `nftables` / `eBPF maps` thay đổi ra sao.
- Can thiệp trực tiếp vào `network namespace` của Pod từ OS Host.

kind là gì? Tại sao nó không phù hợp?

kind = **K**ubernetes **I**N **D**ocker

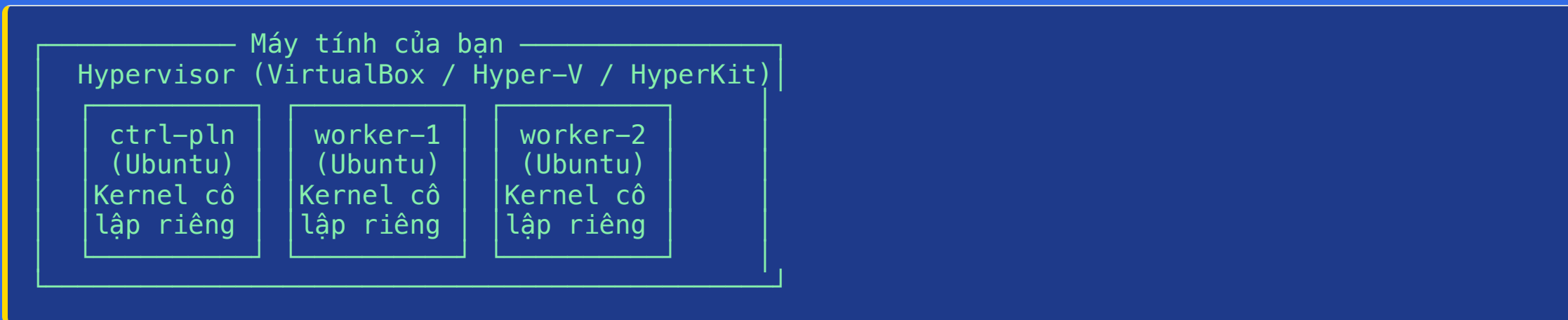


Mỗi "Node" của kind thực chất là một **Container**, không phải một máy ảo thực sự. Chúng **chia sẻ cùng một Linux Kernel** với máy host!

❌ Điểm mù của kind và minikube

Thao tác	kind / minikube	Full VM
Chạy ứng dụng đơn giản	✓	✓
Dùng <code>tcpdump</code> trên card mạng vật lý của Node	✗	✓
Can thiệp <code>ip route</code> , <code>ip link</code> ở mức OS của Node	✗	✓
Quan sát interface VXLAN (<code>flannel.1</code> , <code>vxlan.calico</code>)	✗	✓
Test kernel module (ipvs, nf_conntrack)	✗	✓
Giả lập môi trường Production thực tế	✗	✓

✓ Full VM - Toàn quyền kiểm soát



Mỗi Node là một **máy ảo Ubuntu thực thụ** với **Kernel riêng biệt**. Bạn có thể `ssh` vào và làm mọi thứ giống như một máy chủ vật lý.

Công cụ ảo hóa chúng ta dùng

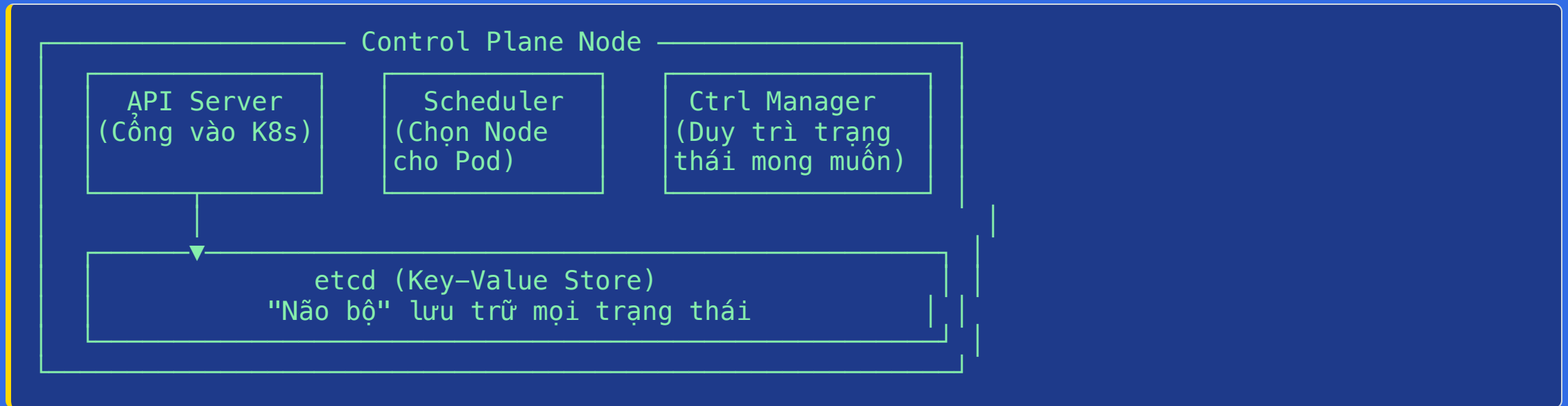
Để khởi động 3 máy ảo tự động (không phải cài tay từng cái), ta dùng:

Hệ điều hành	Công cụ	Hypervisor ngầm
Windows / Linux	Vagrant + VirtualBox	VirtualBox
macOS (Intel)	Vagrant + VirtualBox	VirtualBox
macOS (Apple M-series)	Multipass	HyperKit / QEMU

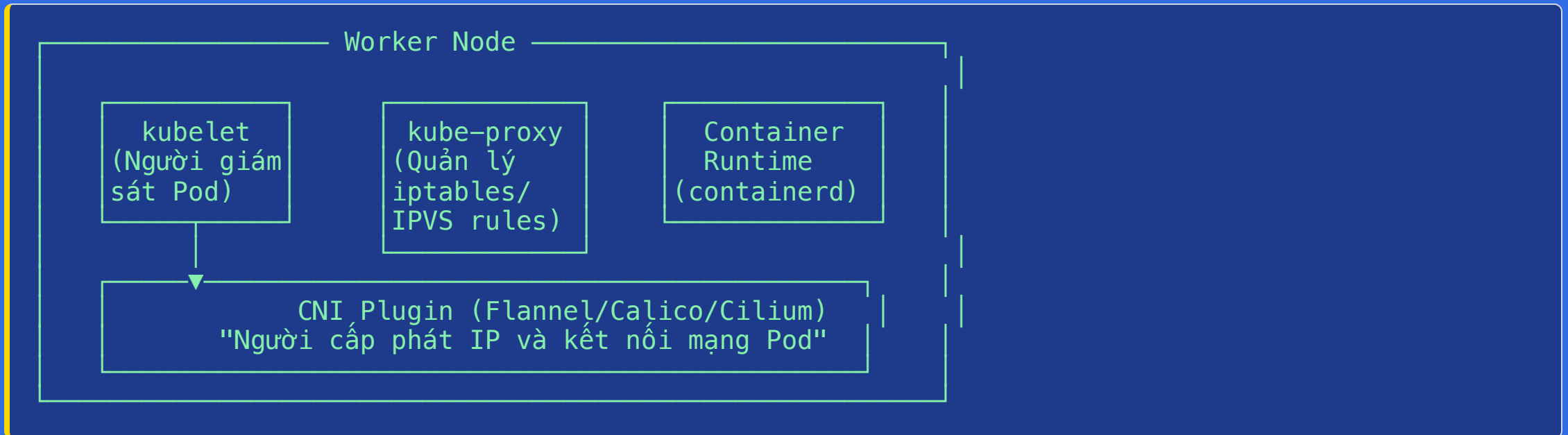
Tại sao không dùng VirtualBox trên Mac M-series?

VirtualBox **không hỗ trợ** chip ARM (Apple Silicon). Multipass là giải pháp Native tối ưu của Canonical dành riêng cho dòng máy này.

⚙️ Kiến trúc K8s Control Plane



⚙️ Kiến trúc K8s Worker Node



🧩 Luồng hoạt động khi tạo một Pod

```
kubectl apply -f pod.yaml
```

↓
[1] API Server nhận yêu cầu và lưu vào etcd

↓
[2] Scheduler chọn Worker Node phù hợp

↓
[3] kubelet trên Worker Node nhận lệnh tạo Pod

↓
[4] kubelet gọi Container Runtime (containerd) tạo container

↓
[5] kubelet gọi CNI Plugin để CẤP PHÁT IP và KẾT NỐI MẠNG

↓
[6] Pod đã chạy và có IP! ✅

? Nếu không có CNI thì sao?

Đây chính là **thí nghiệm đầu tiên** chúng ta sẽ làm trong bài Lab!

Sau khi `kubeadm init`, chúng ta **cố tình KHÔNG cài CNI** và quan sát:

```
$ kubectl get nodes
NAME             STATUS    ROLES    AGE
controlplane     NotReady  control-plane  2m # ← NotReady!
worker1          NotReady  <none>       1m # ← NotReady!
worker2          NotReady  <none>       1m # ← NotReady!

$ kubectl get pods -n kube-system
NAME             READY   STATUS    RESTARTS
coredns-xxx      0/1     Pending   0 # ← Pending!
```

Thiếu CNI = Không có mạng = Node `NotReady` = CoreDNS `Pending`.

Tóm lại

- **kind / minikube:** Tuyệt vời để dev ứng dụng nhanh. **Không phù hợp** để học sâu về networking.
- **Full VM (Vagrant / Multipass):** Môi trường **giống Production thực tế nhất** trên máy tính cá nhân. Toàn quyền can thiệp kernel.
- **K8s Control Plane** gồm: API Server, Scheduler, Controller Manager, etcd.
- **Worker Node** gồm: kubelet, kube-proxy, Container Runtime, và **CNI Plugin** (điều chúng ta sẽ mổ xẻ xuyên suốt khóa học).

👉 Bước tiếp theo: Bài Lab

Mở thư mục `lab-module0/` và làm theo hướng dẫn phù hợp với hệ điều hành của bạn:

- **Vagrant (Windows/Linux/macOS Intel):** `lab-module0-guide.md`
- **Multipass (macOS Apple Silicon):** `lab-module0-macos-guide.md`